# Chapter 4: Traps and System Calls

xv6 RISC-V Study Notes

*Based on Cox, Kaashoek & Morris* — xv6: a simple, Unix-like teaching OS *(rev. 3, 2022)*

## 1 Big Picture: What Problem Does This Chapter Solve?

### 1.1 The Core Challenge

A running user program must sometimes ask the kernel to do things it cannot do itself — read from disk, create a process, write to a network socket. Equally, the hardware may need to interrupt the CPU at any moment (e.g., when a disk finishes a read). And occasionally a program does something illegal (divide by zero, invalid address access).

**All three situations require the same solution:** the CPU must abandon what it is currently doing, transfer control to the kernel in a controlled, secure way, let the kernel handle the event, and then (usually) resume normal execution.

### 1.2 The Three Trap Triggers

| Trigger | Example | Origin |
|---|---|---|
| System call | `ecall` instruction | User program |
| Exception | Divide by zero, illegal address access | User or kernel |
| Device interrupt | Disk completes I/O, timer fires | Hardware device |

xv6 uses the word **trap** as a unified term for all three.

### 1.3 Why Traps Must Go Through the Kernel

- **System calls:** Only the kernel has the privilege to perform protected operations.
- **Interrupts:** Isolation demands that only the kernel talks to devices; the kernel then multiplexes device access across processes.
- **Exceptions:** xv6 kills any user process that causes an exception (exceptions in the kernel cause a `panic`).

### 1.4 The Four Stages of Every Trap

xv6 trap handling always follows the same four-stage pipeline:

1. **Hardware action** — the RISC-V CPU atomically saves minimal state and redirects the PC.
2. **Assembly vector** — a short piece of assembly (the *handler vector*) prepares a safe environment for C code.
3. **C trap handler** — decides what kind of trap occurred and dispatches to the right service routine.
4. **Service routine** — actually implements the system call, handles the interrupt, or deals with the exception.

**Section Summary**

Traps are the universal mechanism by which the CPU transitions from normal execution into the kernel. They are triggered by system calls, exceptions, or device interrupts. Every

trap follows the same four-stage pipeline: hardware action → assembly vector → C handler → service routine.

## 2 Core Mechanisms Introduced

### 2.1 Key Concepts at a Glance

| Mechanism | Purpose |
|---|---|
| `stvec` | Supervisor Trap VECtor register. Kernel writes the address of its trap handler here; CPU jumps to it on any trap. |
| `sepc` | Saved Exception Program Counter. RISC-V saves `pc` here when a trap occurs. |
| `scause` | Describes the reason for the trap (a numeric code). |
| `sscratch` | Scratch register available to the trap handler before it can save user registers. |
| `sstatus` | Status register: SIE bit enables/disables device interrupts; SPP bit records whether the trap came from user or supervisor mode. |
| `sret` | Return-from-trap instruction; copies `sepc` back to `pc` and restores privilege mode. |
| `trampoline page` | A special page mapped at the same virtual address in *both* user and kernel page tables, containing the user-space trap entry/exit asm (`uservec`/`userret`). |
| `trapframe` | Per-process kernel-allocated page (mapped in user address space) that holds the saved user registers while the kernel runs. |

**Section Summary**

The six RISC-V control registers (`stvec`, `sepc`, `scause`, `sscratch`, `sstatus`, `sret`) plus the software-defined trampoline page and trapframe structure form the complete infrastructure for trap handling.

## 3 Hardware vs. Kernel Responsibilities

**Hardware Responsibilities**

**When a trap fires, the RISC-V CPU automatically does (in order):**

1. If the trap is a device interrupt and `sstatus.SIE` is clear, abort (do nothing).
2. Clear `sstatus.SIE` — disable further device interrupts.
3. Copy current `pc` to `sepc`.
4. Save current privilege mode in `sstatus.SPP`.
5. Set `scause` to the trap's cause code.
6. Switch mode to **supervisor** (kernel mode).
7. Copy `stvec` → `pc` (jump to the trap handler).
8. Start executing at the new `pc`.

**Crucially, the hardware does NOT:**

- Switch page tables (still using whatever page table was active).
- Switch to a kernel stack.
- Save any general-purpose registers (only `pc` is implicitly saved via `sepc`).

The CPU deliberately does minimal work to give the kernel maximum flexibility.

**Kernel Responsibilities**

**The kernel must then do everything the hardware left undone:**

1. Save all 32 user general-purpose registers to the trapframe.
2. Switch page tables from user to kernel.
3. Switch to the kernel stack.
4. Identify the trap cause (`scause`) and dispatch to the right handler (system call, device interrupt, or exception).
5. (System calls) Retrieve arguments from the trapframe, execute the service, write return value back to the trapframe.
6. (Interrupts) Run the device driver.
7. (Exceptions) Kill the offending process (user space) or panic (kernel space).
8. On return: restore user registers from the trapframe, switch back to user page table, execute `sret`.

**Section Summary**

The hardware/kernel split is intentional: RISC-V does the bare minimum (save PC, set mode, jump to handler) so the kernel retains full control over security-critical decisions like page table switching and stack selection. The kernel must save all register state, switch address spaces, and orchestrate the entire service and return path.

## 4 Full Trap Lifecycle: Step-by-Step

xv6 maintains *separate code paths* for three cases. The following walks through each.

### 4.1 Case 1 — Trap from User Space

**Entry path:** `uservec` (trampoline.S:21) → `usertrap` (trap.c:37)
**Exit path:** `usertrapret` (trap.c:90) → `userret` (trampoline.S:101)

**Step-by-step:**

1. **[HW] Trap fires.** CPU saves `pc` → `sepc`, sets `scause`, switches to supervisor mode, jumps to `stvec` (which points to the trampoline's `uservec`).
2. **[uservec] Swap a0 / sscratch.** `uservec` uses the `csrw` instruction to exchange `a0` with `sscratch`. This frees one register (`a0`) to work with. `sscratch` had been pre-loaded (by the kernel before returning to user space) with the address of the process's **trapframe**.
3. **[uservec] Save all 32 user registers.** `uservec` loads the trapframe address into `a0` and saves all 32 user registers (including the real `a0` recovered from `sscratch`) into the trapframe.
4. **[uservec] Switch page tables and jump to usertrap.** The trapframe also contains the kernel page table address, kernel stack pointer, and `usertrap`'s address. `uservec`

loads these, switches `satp` to the kernel page table, and calls `usertrap`. (This works because the trampoline page is mapped at the *same* virtual address in both page tables.)

5. **[usertrap] Redirect stvec.** `usertrap` immediately changes `stvec` to point to `kernelvec` so that any trap that occurs *while in the kernel* is handled correctly.
6. **[usertrap] Save sepc.** `usertrap` saves `sepc` into the trapframe (because a subsequent `yield()` call could overwrite it).
7. **[usertrap] Dispatch on cause.**
   - `scause == 8` (ecall): it's a **system call** → call `syscall()`, also add 4 to saved `sepc` so `sret` returns to the instruction *after* `ecall`.
   - Device interrupt → call `devintr()`.
   - Anything else → **exception**: kill the process (`p->killed = 1`).
8. **[syscall()] Retrieve arguments and dispatch.** `syscall()` reads the system call number from `a7` in the trapframe, uses it to index the `syscalls[]` function pointer table, calls the appropriate handler (e.g., `sys_exec`), and stores the return value back in `p->trapframe->a0`.
9. **[usertrap end] Check for kill/yield.** Before returning, `usertrap` checks if the process was killed or if a timer interrupt should cause a `yield()`.
10. **[usertrapret] Prepare control registers for user re-entry.** `usertrapret` restores `stvec` to `uservec`, refreshes trapframe fields (kernel stack, kernel page table, `usertrap` address), loads the saved `sepc` back into the `sepc` register, and calls `userret` on the trampoline page.
11. **[userret] Switch back and restore registers.** `userret` switches `satp` back to the user page table, restores all 32 user registers from the trapframe, and executes `sret` to return to user space at the saved PC.

### *4.1.1 Why the Trampoline Page Exists*

**Foundational Concept**

At the moment a user-space trap fires, the CPU is still using the **user page table** (RISC-V does not switch page tables on trap entry). The code in `stvec` must therefore be mapped in the user page table. But after saving registers, that code needs to switch to the kernel page table—and must still be reachable after the switch. The only way to satisfy both constraints is to map the same physical page at the same virtual address in *both* tables. This page is the **trampoline**.

## 4.2 Case 2 — Trap from Kernel Space

**Entry:** `kernelvec` (kernelvec.S:12) → `kerneltrap` (trap.c:135)

**Key differences from user-space traps:**

1. **[HW] Trap fires.** Same hardware actions as always, but `stvec` already points to `kernelvec` (set by `usertrap` on kernel entry).
2. **[kernelvec] Save registers on the kernel stack.** Since the kernel is already running with the kernel page table and a valid kernel stack, `kernelvec` pushes all 32 registers directly onto the *interrupted kernel thread's* stack. No page table switch is needed.

3. **[kerneltrap] Dispatch.**
   - Device interrupt → `devintr()`.
   - Any other exception → fatal; call `panic()` and halt.
   - Timer interrupt → `yield()` if a process kernel thread is running.
4. **[kerneltrap end] Restore and return.** `kerneltrap` restores `sepc` and `sstatus` (a `yield()` may have disturbed them), then returns to `kernelvec`.
5. **[kernelvec] Pop registers and sret.** `kernelvec` pops all saved registers from the kernel stack and executes `sret`, resuming the interrupted kernel code.

**Implementation Detail**

**The stvec window:** When `usertrap` first enters the kernel, `stvec` is still pointing at `uservec` for a brief window before it is updated to `kernelvec`. RISC-V guarantees that interrupts are disabled when entering a trap, so no device interrupt can arrive during this window. xv6 does not enable interrupts again until after `stvec` is safely set to `kernelvec`.

### 4.3 Case 3 — System Call Argument Passing

1. User code places arguments in registers `a0`–`a5` per the RISC-V C calling convention, and the system call number in `a7`.
2. `ecall` fires; `uservec` saves all registers into the trapframe.
3. `syscall()` reads `a7` from the trapframe to look up the handler.
4. Kernel helper functions `argint()`, `argaddr()`, `argfd()` retrieve the $n$-th argument from the trapframe.
5. For pointer arguments: `fetchstr()` / `copyinstr()` safely walk the *user* page table to copy strings into kernel memory. `copyout()` does the reverse.
6. The return value is written to `p->trapframe->a0`, which `userret` will restore to the user's `a0`.

**Foundational Concept**

**Why pointers from user space are dangerous:** A malicious user program may pass a pointer that (a) points into kernel memory, or (b) is mapped differently in the kernel's page table. The kernel *must not* dereference user pointers directly using ordinary load/store instructions. Instead, it uses `copyinstr` / `copyout` which explicitly walk the *user* page table, validate addresses with `walkaddr`, and then copy data safely.

**Section Summary**

There are three trap paths: (1) user-space traps use the trampoline + trapframe dance to safely switch address spaces; (2) kernel-space traps are simpler because the kernel stack and page table are already set up; (3) system call argument retrieval goes through trapframe registers and requires special safe-copy helpers for pointer arguments.

# 5 Page-Fault Exceptions and Advanced VM Tricks

## 5.1 xv6's Minimalist Response

In xv6, any exception in user space kills the process; any exception in the kernel causes a `panic`. This is safe but not sophisticated.

## 5.2 What Real Systems Do with Page Faults

Page-fault exceptions are the foundation for several powerful virtual-memory features. The `scause` register encodes the fault type; `stval` holds the faulting virtual address.

| Technique | Mechanism |
|---|---|
| Copy-on-Write (COW) fork | Parent and child share physical pages mapped read-only. On a write, a page-fault allocates a new page, copies content, and updates the PTE to allow writes. `fork()` becomes cheap; copying is deferred until actually necessary. |
| Lazy allocation | `sbrk()` only records the new size; it does not allocate pages. On the first access, a page-fault handler allocates and maps the page on demand. |
| Demand paging | `exec` maps PTEs as invalid; pages are loaded from disk only when first accessed via a page fault. Reduces startup latency for large programs. |
| Paging to disk | Pages not recently used are evicted to a *paging area* on disk and their PTEs marked invalid. A page fault on access pages them back in. |
| Memory-mapped files | File contents are mapped into the address space; faults load the relevant disk block on demand. |

**Foundational Concept**

**The key insight:** By intercepting page faults, the kernel can *lie to the application* about the state of memory (it doesn't have to be resident, writable, or even exist yet). This is the fundamental mechanism behind lazy memory management in all production OS kernels.

**Section Summary**

xv6 uses a trivially simple exception policy (kill or panic), but the chapter introduces how real kernels exploit page-fault exceptions for COW fork, lazy allocation, demand paging, and disk paging — all transparent to user applications. These are all variations on the same theme: the trap handler allocates/fixes memory state and resumes the faulting instruction.

# 6 Foundational vs. Implementation Details

## 6.1 Foundational Concepts (Must Understand)

**Foundational Concepts**

1. **Traps are the kernel's boundary crossing mechanism.** Every interaction between a user process and the hardware or kernel goes through a trap. Understanding traps means understanding how the OS enforces isolation.
2. **The hardware–software contract.** RISC-V does the minimum (save PC, set mode, jump). The kernel is responsible for everything else. This split is a deliberate design philosophy.
3. **The trampoline page solves the page-table bootstrapping problem.** When a user-space trap fires, the MMU is still using the user page table. The trap handler must be mapped there. But the handler must also switch to the kernel page table. The trampoline, mapped identically in both, is the solution.
4. **Transparent trap handling.** Normal code has no idea a trap occurred. The entire mechanism is designed so that execution can resume exactly where it left off, with all registers and state restored.
5. **User pointers cannot be trusted.** The kernel must validate and copy user-supplied pointers/data using dedicated helper functions; direct dereference of user pointers is a security flaw.
6. **Page faults are the hook for lazy virtual memory.** The OS can defer physical-memory work until it is actually needed by intercepting page faults. This pattern underlies COW fork, lazy allocation, demand paging, and swapping.

## 6.2 Implementation Details (Good to Know)

**Implementation Detail**

- `sscratch` is pre-loaded with the trapframe address by `usertrapret` before returning to user space.
- `TRAPFRAME` is mapped just below `TRAMPOLINE` in every user address space.
- `sepc` is saved into the trapframe by `usertrap` because a `yield()` call could corrupt the hardware `sepc` register.
- System call numbers are passed in `a7`; return values in `a0` (RISC-V C calling convention).
- The `sepc` for system calls is incremented by 4 to skip past the `ecall` instruction.
- `kernelvec` saves registers on the kernel stack (not a trapframe), because the interrupted state belongs to that kernel thread.
- `kerneltrap` saves/restores `sepc` and `sstatus` because `yield()` may disturb them.
- xv6 avoids mapping kernel memory into the user page table (unlike some kernels) to reduce the risk of inadvertent user-pointer dereference bugs and to side-step address-space overlap issues.

## 7 Quick-Reference: CSR Registers

| Register | Role | Who writes / when |
|---|---|---|
| `stvec` | Trap handler address | Kernel: `uservec` addr before returning to user; `kernelvec` addr on kernel entry |
| `sepc` | Saved user/kernel PC | HW on trap; kernel adds 4 for system calls |
| `scause` | Trap cause code | HW on trap (read by kernel) |
| `sscratch` | Scratch / trapframe pointer | Kernel pre-loads before user return; `uservec` swaps with `a0` |
| `sstatus` | Interrupt enable; mode flag | HW clears SIE on trap; kernel manages SIE for re-enabling |
| `satp` | Page table root | `uservec` switches user→kernel; `userret` switches kernel→user |
| `stval` | Faulting address (page faults) | HW on page-fault (read by kernel handler) |

## 8 Overall Chapter Summary

**Chapter 4 — Complete Summary**

Chapter 4 answers the question: *how does the OS regain control of the CPU without breaking the illusion of continuous user execution?*
The answer is the **trap mechanism**, a collaborative protocol between RISC-V hardware and the xv6 kernel:

- **Hardware** atomically saves the PC, encodes the cause, switches to supervisor mode, and jumps to `stvec`.
- **Kernel assembly** (trampoline/`kernelvec`) saves all registers and switches address spaces.
- **Kernel C code** (`usertrap`/`kerneltrap`/`syscall`) identifies and services the trap.
- **Kernel assembly** (`userret`/`kernelvec`) restores all registers, switches address spaces back, and executes `sret`.

Three separate paths exist for user-space traps, kernel-space traps, and timer interrupts, each optimised for its context. The trampoline and trapframe are the two key data structures that make the user-space path possible.
Finally, the chapter uses page-fault exceptions to motivate advanced virtual-memory techniques (COW fork, lazy allocation, demand paging, disk paging) that production kernels implement by extending the same trap infrastructure.